

TECHNICAL DOCUMENTATION

ShowSzn

A BookMyShow-style ticket booking platform — architecture, data flow, and build log

Stack: Spring Boot 4 (Java 21) · MySQL 8.4 · Redis 7 · React 19 · TypeScript · Clerk

Generated: August 2026

Repository: ~/projects/showszn

Contents

1. Project Overview
2. Repository Structure
3. Database Schema
4. Authentication & Authorization Flow
5. Booking & Seat-Locking Flow
6. Caching Strategy
7. API Reference
8. Frontend Data Flow
9. Build Log: Decisions & Bugs Fixed
10. Environment & Setup
11. Known Limitations & Next Steps

1. Project Overview

ShowSzn is a full-stack movie ticket booking platform modeled on BookMyShow: browse movies by city, view showtimes across multiple cinema chains, pick seats on a real seat map, and book with a live (Redis-backed) seat-locking flow. It also has a lightweight Events section and an admin panel for managing the catalog.

Layer	Technology	Notes
Backend	Spring Boot 4.1.0, Java 21	Built with Spring Data JPA, Spring Security (OAuth2 resource server), Spring Cache
Database	MySQL 8.4	Migrated from PostgreSQL mid-project; schema managed by Flyway
Cache / Locking	Redis 7	Used for both HTTP response caching and seat-hold locks (SETNX pattern)
Auth	Clerk	Frontend hosted sign-in; backend verifies JWTs against Clerk's JWKS endpoint
Frontend	React 19 + TypeScript + Vite	TanStack Query for server state, React Router for navigation, Tailwind CSS 4 for styling
Infra (local)	Docker Compose	Runs MySQL + Redis; app runs via <code>mvnw spring-boot:run / npm run dev</code>

The build order across the project was: scaffolding → data model → auth → browse (movies/shows) → admin panel → geolocation + trailers + UI polish → scalability (pagination, caching, more cities) → MySQL migration + realistic multi-mall venues + corrected movie/trailer catalog → full booking & seat-selection flow → site structure (logo, search, footer, category nav, events, banners).

2. Repository Structure

```
showszn/
├─ docker-compose.yml      # MySQL + Redis for local dev
├─ backend/
│  └─ pom.xml
│     └─ src/main/
│        └─ java/com/showsxn/backend/
│           ├── catalog/    # City, Venue, Screen, Seat, Movie, Show, ShowSeat
│           ├── user/       # AppUser, sync from Clerk JWT
│           ├── security/   # JWT verification + ROLE_ADMIN converter
│           ├── admin/      # Admin CRUD APIs (movies/venues/screens/shows)
│           ├── booking/    # Booking, seat locking, hold/confirm flow
│           ├── event/      # Lightweight curated events
│           ├── cache/      # Redis cache serializer config
│           └─ common/      # Health check
│        └─ resources/
│           ├── application.yml
│           └─ db/migration/ # Flyway SQL migrations (V1-V5)
└─ frontend/
   └─ src/
      ├── pages/           # Route-level components
      ├── pages/admin/     # Admin panel pages
      ├── components/      # Header, Footer, SearchBar, MovieCard, etc.
      ├── lib/             # axios client, TanStack Query hooks, helpers
      ├── context/         # CityContext (selected city, persisted)
      └─ types.ts          # Shared TypeScript interfaces
```

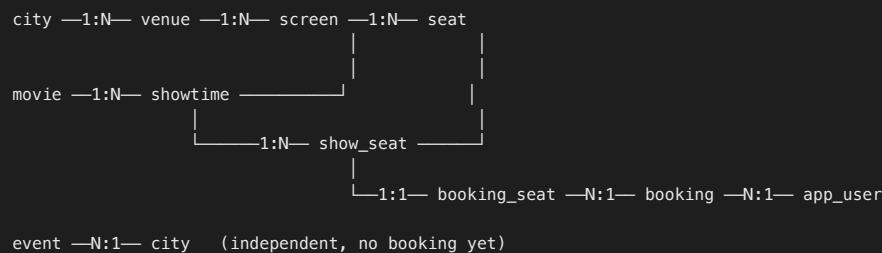
Backend package responsibilities

Package	Responsibility
catalog	Core domain: cities, venues, screens, seats, movies, shows, and per-show seat inventory (<code>ShowSeat</code>). Owns the public browse API (<code>CatalogController</code>) and search.
user	<code>AppUser</code> entity synced from Clerk JWT claims on first authenticated request; bootstrap-admin-by-email mechanism.
security	<code>SecurityConfig</code> wires the OAuth2 resource server (JWT validation against Clerk's JWKS) and route authorization rules; <code>ClerkJwtAuthenticationConverter</code> enriches the authenticated principal with <code>ROLE_ADMIN</code> by looking up the DB.
admin	CRUD endpoints for movies, venues, screens, and shows, gated by <code>ROLE_ADMIN</code> . Screen creation auto-generates the seat map; show creation auto-generates <code>ShowSeat</code> inventory.
booking	The seat-selection → hold → confirm flow. <code>SeatLockService</code> wraps Redis; <code>BookingService</code> owns the transactional DB writes; <code>SeatMapService</code> serves the public seat map.
event	Minimal read-only events feature (city-filtered, no booking yet).
cache	Custom Redis cache configuration using per-cache explicitly-typed Jackson serializers (see §6).

Frontend structure

Path	Responsibility
lib/api.ts	Shared axios instance, <code>baseUrl</code> <code>/api</code> (proxied to backend by Vite in dev).
lib/useAuthTokenInjector.ts	Hook that attaches the Clerk session JWT to every outgoing axios request via an interceptor.
lib/queries.ts , adminQueries.ts , bookingQueries.ts	TanStack Query hooks grouped by domain (public browse, admin CRUD, booking).
context/CityContext.tsx	Selected city, persisted to <code>localStorage</code> , with geolocation-based auto-detect on first visit.
pages/	One component per route: Home, MovieDetail, SeatSelection, Events, ComingSoon (Plays/Sports/Activities), Admin/.

3. Database Schema



Note on naming: the show table is called `showtime`, not `show` — `SHOW` is a reserved word in MySQL and would require escaping in every query. Renaming avoided that entirely (the JPA entity class is still named `Show`; only the physical table name changed via `@Table(name = "showtime")`).

Catalog tables

Table	Key columns	Purpose
city	name, state, slug, latitude, longitude	10 seeded cities. Lat/lng used for geolocation-based auto-selection on the frontend.
venue	city_id (FK), name, address, slug	Cinema/mall. 35 seeded (3–4 per city, real chain names: PVR, INOX, Cinapolis, etc.)
screen	venue_id (FK), name	Auditorium within a venue. 2 per venue (70 total).
seat	screen_id (FK), row_label, seat_number, seat_type	Static physical seat map. 114 seats/screen: rows A–C Regular, D–G Premium, H–I Recliner.
movie	title, slug, duration_minutes, language, genre, release_date, poster_url, trailer_url, censor_rating	8 seeded movies (real, openly-licensed short films used as trailer content — see §9).
showtime	movie_id (FK), screen_id (FK), start_time, end_time, base_price	A movie playing on a specific screen at a specific time. 2,240 seeded (8 movies × 70 screens × 2 days × 2 slots).
show_seat	show_id (FK), seat_id (FK), price, status	Bookable inventory — one row per seat per showtime. status ∈ AVAILABLE / BOOKED (LOCKED is Redis-only, see §5). ~255K rows seeded.

User & booking tables

Table	Key columns	Purpose
app_user	clerk_user_id (unique), email, name, is_admin	Synced lazily from the Clerk JWT on first authenticated request.
booking	booking_reference (unique), user_id (FK), show_id (FK), status, total_amount	One row per confirmed booking. status is CONFIRMED in the current mock-payment flow.
booking_seat	booking_id (FK), show_seat_id (FK, unique), price	Line items of a booking. Unique constraint on show_seat_id prevents the same seat being booked twice at the DB level, as a second line of defense behind the Redis lock.

Events table

Table	Key columns	Purpose
event	city_id (FK), title, category, venue_name (free text), event_date, price, image_url	Browsable only — no seat/booking model yet. 10 seeded across cities.

4. Authentication & Authorization Flow

- 1 User clicks "Sign in" → Clerk's hosted modal handles sign-up/sign-in (frontend only, via `@clerk/clerk-react`).
- 2 Clerk issues a session JWT. `useAuthTokenInjector` attaches it as `Authorization: Bearer <token>` on every axios request.
- 3 Spring Security's OAuth2 resource server validates the JWT against Clerk's JWKS (`{CLERK_ISSUER}/.well-known/openid-configuration`), lazily and cached.
- 4 `ClerkJwtAuthenticationConverter` runs on every authenticated request: looks up `app_user` by the `sub` claim, and if `is_admin = true`, grants the `ROLE_ADMIN` authority.
- 5 On endpoints that need a persisted user (e.g. `/api/me`, booking confirm), `UserSyncService.syncFromJwt()` creates or updates the `app_user` row from the `sub` / `email` / `name` claims. the JWT's
- 6 **Bootstrap admins:** the `ADMIN_EMAILS` env var (comma-separated) is checked on every sync — any matching email is auto-promoted to admin, even on a fresh database. No manual SQL needed.

Route authorization rules (SecurityConfig)

Pattern	Rule
<code>/api/public/**</code>	Open, no auth required
<code>/api/admin/**</code>	Requires <code>ROLE_ADMIN</code>
Everything else (<code>/api/me</code> , <code>/api/bookings/**</code>)	Requires any authenticated user

Important Clerk configuration: the JWT's default claims don't include email/name. A custom session token template must be configured in the Clerk dashboard (Configure → Sessions → Customize session token) with:

```
{ "email": "{{user.primary_email_address}}", "name": "{{user.full_name}}" }
```

5. Booking & Seat-Locking Flow

The seat map is public (no auth needed to view). Holding and confirming seats requires authentication. Redis is the source of truth for temporary holds — the database's `show_seat.status` only ever transitions `AVAILABLE` → `BOOKED`, never to an intermediate "locked" state, which avoids needing a background cleanup job for expired holds (Redis TTL handles that automatically).



Why Redis SETNX and not a DB row lock?

Using Redis avoids two problems a DB-only design would have: (a) a hold that's never confirmed (user closes the tab) would leave seats stuck in a "locked" state forever without a cleanup job, and (b) checking-then-setting seat status in the DB under concurrent requests needs pessimistic locking or careful transaction isolation to avoid a race where two users both see a seat as available. Redis's `SETNX ... EX` is atomic and self-expiring, which sidesteps both.

6. Caching Strategy

Two hot, read-heavy, rarely-changing public endpoints are cached in Redis via Spring's `@Cacheable`: the city list (`cities` cache) and "movies playing in city X" (`moviesByCity` cache, keyed by city ID). TTL is 5 minutes. Cache is evicted (`@CacheEvict(allEntries = true)`) whenever an admin creates/updates/deletes a movie, or creates a show — either could change what's "now showing" in a city.

Bug found & fixed: Spring Boot's default Redis cache serializer (`GenericJacksonJsonRedisSerializer`) loses the element type of generic collections like `List<CityResponse>` on deserialize — Jackson has no way to know the target type from a cache lookup alone, so it falls back to `List<LinkedHashMap>`. This threw `HttpMessageNotWritableException` when serving a cache hit, which — critically — doesn't surface as a 500. Spring's error-handling re-dispatches to `/error` internally, which isn't covered by the `/api/public/**` permit-all rule, so it fell through to "authentication required" and returned a misleading **401** instead. Fixed by registering a per-cache-name `RedisCacheConfiguration` with an explicitly-typed `JacksonJsonRedisSerializer` (concrete `JavaType` per cache, no generic type erasure possible). See `cache/CacheConfig.java`.

This project runs on a very recent Spring Boot 4 / Jackson 3 stack, where Jackson's Maven coordinates moved from `com.fasterxml.jackson.*` to `tools.jackson.*`. Spring Data Redis's classic `GenericJackson2JsonRedisSerializer` (the "2" suffix) is hardcoded to the old Jackson 2 API and isn't on the classpath at all in this stack — the Jackson-3-compatible replacement is `GenericJacksonJsonRedisSerializer` (no "2"). This is worth knowing if the Jackson/Redis dependency versions ever get bumped again.

7. API Reference

Public — Catalog & Search

Endpoint	Description
GET /api/public/cities	All cities (cached)
GET /api/public/movies?cityId=	Movies with an upcoming show in the given city (cached)
GET /api/public/movies/{id}	Movie detail incl. trailer URL
GET /api/public/movies/{id}/shows?cityId=&date=	Showtimes for a movie in a city on a date, grouped by venue on the client
GET /api/public/events?cityId=	Curated events, optionally filtered by city
GET /api/public/search?q=	Combined movie-title and venue-name search
GET /api/public/health	Liveness check

Public — Seat Map

Endpoint	Description
GET /api/public/shows/{showId}/seat-map	Full seat layout for a show with live AVAILABLE/LOCKED/BOOKED status

Authenticated — Me & Bookings

Endpoint	Description
GET /api/me	Current user profile (syncs from JWT if first call), incl. isAdmin
POST /api/bookings/hold	Acquire a 5-minute Redis lock on selected seats
POST /api/bookings/confirm	Finalize a hold into a real booking
GET /api/bookings/{reference}	Fetch one booking (owner only)
GET /api/bookings	List the current user's bookings

Admin — requires ROLE_ADMIN

Endpoint	Description
GET /api/admin/movies	Paginated movie list (?page=&size=)
POST /api/admin/movies	Create a movie
PUT /api/admin/movies/{id}	Update a movie
DELETE /api/admin/movies/{id}	Delete a movie
GET / POST /api/admin/venues	Paginated list / create a venue
GET / POST /api/admin/venues/{venueId}/screens	List / create a screen (auto-generates its 114-seat layout)
GET / POST /api/admin/shows	Paginated list / create a showtime (auto-generates its show_seat inventory)

8. Frontend Data Flow

Routing table

Path	Component
/	HomePage — banner carousel, "now showing" grid, trending events strip
/movies/:movieId	MovieDetailPage — trailer modal, date tabs, showtimes by venue
/shows/:showId	SeatSelectionPage — seat map, hold, mock pay, confirmation
/events	EventsPage — real, city-filtered event cards
/plays , /sports , /activities	ComingSoonPage (shared placeholder component)
/admin/*	AdminLayout + nested tabs (movies/venues/screens/shows), gated client-side by <code>me.isAdmin</code>

State management patterns

- **Server state:** entirely TanStack Query. Every domain has its own hooks file (`queries.ts` for public browse, `adminQueries.ts` , `bookingQueries.ts`). Mutations invalidate the relevant query keys on success rather than manually patching cache.
- **Selected city:** `CityContext` , persisted to `localStorage` under `showszn.selectedCityId` . On first visit with no stored preference, `CitySelector` tries `navigator.geolocation` and picks the nearest seeded city via Haversine distance (`lib/geo.ts`); falls back to the first city if denied or unsupported.
- **Auth token:** `useAuthTokenInjector` is called once near the app root; it registers an axios request interceptor that calls Clerk's `getToken()` and attaches it as a Bearer header on every request (harmless no-op on public endpoints).
- **Pagination:** admin list pages carry their own `page` state and read Spring's `Page<T>` response shape (`content` , `totalPages` , etc.) via a shared `Pagination` component. Dropdowns that need the *full* list (e.g. picking a venue when creating a show) use separate `useAllAdminVenues` / `useAllAdminMovies` hooks that request a large page size rather than reusing the paginated hook.

9. Build Log: Decisions & Bugs Fixed

In build order. Each entry is a real issue hit during development, not a hypothetical.

Movie & trailer catalog

The movie catalog uses real, verified, openly-licensed short films (Blender Foundation / Blender Studio open movies) as trailer content, since fictional movies can't have real trailers. Each was verified via YouTube's oEmbed API before use — title, author, and license confirmed, not guessed.

Movie	Genre	Trailer source
Big Buck Bunny	Comedy	Blender Foundation
Sintel	Fantasy	Blender Foundation
Tears of Steel	Sci-Fi	Blender Foundation
Cosmos Laundromat	Sci-Fi Comedy	Blender Foundation
Spring	Fantasy	Blender Studio
Elephants Dream	Sci-Fi	Blender Foundation (the first-ever open movie)
Caminandes: Llama Drama	Comedy	Blender Foundation (CC BY 3.0 reupload)
Charge	Action	Blender Studio

Earlier in development, movies had fictional titles paired arbitrarily with these trailers (e.g. a Hindi thriller playing a cute rabbit comedy). This was corrected by renaming every movie to match what its trailer actually shows — not a data bug, a content design fix.

Recurring bug pattern: lazy-loaded associations masked as 401

Found **three separate times** in different services (catalog shows, admin venues, admin shows): accessing a lazily-fetched JPA association (e.g. `show.getScreen().getVenue().getName()`) outside an open Hibernate session throws `LazyInitializationException`. Because this happens during JSON response serialization — after the controller method has already returned — Spring's error-dispatch mechanism re-enters the servlet pipeline via `/error`, which isn't covered by the security config's permit-all rule, so it returns **401 Unauthorized** instead of a 500. This is deeply misleading: it looks like an auth bug when it's actually a data-access bug.

Fix pattern used throughout: add `JOIN FETCH` to the repository query so every association the DTO mapper touches is loaded in the same query, eliminating the lazy proxy entirely rather than trying to keep the transaction open longer.

MySQL migration

Issue	Fix
<code>SHOW</code> is a MySQL reserved word	Renamed the table to <code>showtime</code> (entity class stays <code>Show</code> , only <code>@Table(name=...)</code> changed)
No <code>generate_series()</code> in MySQL	Used <code>WITH RECURSIVE</code> CTEs for seat-number generation
<code>WITH RECURSIVE ... INSERT INTO ...</code> syntax error	MySQL requires the CTE <i>after</i> <code>INSERT INTO table (cols)</code> , not before the whole statement (unlike leading a plain <code>SELECT</code>)
Stale target/classes resources	Old Postgres-syntax migration files were still copied into <code>target/classes</code> after being deleted from source — required <code>mvn clean</code> , not just <code>compile</code>
Failed migration left Flyway's schema history dirty	Dropped and recreated the schema rather than attempting <code>flyway repair</code> (acceptable since there was no real user data yet)

Redis / Jackson 3

See §6 for the full explanation — `GenericJackson2JsonRedisSerializer` doesn't exist on this stack's classpath (Jackson 3, not 2); its replacement is `GenericJacksonJsonRedisSerializer`, and it needs an explicit `JavaType` per cache to avoid generic-collection type loss on deserialize.

Scaffolding-era fixes

Issue	Fix
Spring Boot parent version <code>4.1.0.RELEASE</code> not found	Spring Boot 3+ dropped the <code>.RELEASE</code> suffix from version strings; correct version was <code>4.1.0</code>
Docker MySQL 8.4 wouldn't start with <code>--default-authentication-plugin</code> flag	That flag was removed in MySQL 8.4 (modern connectors support <code>caching_sha2_password</code> natively) — dropped it from <code>docker-compose.yml</code>

10. Environment & Setup

Required environment variables (backend/.env)

Variable	Purpose
CLERK_ISSUER	Clerk Frontend API URL, e.g. <code>https://your-app.clerk.accounts.dev</code> — used as the JWT issuer for validation
ADMIN_EMAILS	Comma-separated emails auto-promoted to admin on sign-in
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET	Placeholder — not yet wired up (see §11)

Required environment variables (frontend/.env)

Variable	Purpose
VITE_CLERK_PUBLISHABLE_KEY	Clerk publishable key (safe to expose client-side)

Local run

```
# 1. Start MySQL + Redis
docker compose up -d

# 2. Backend (runs Flyway migrations automatically on boot)
cd backend
./mvnw spring-boot:run

# 3. Frontend
cd frontend
npm install
npm run dev
```

Frontend dev server: `http://localhost:5173` (Vite proxies `/api` to the backend on `:8080`). Backend: `http://localhost:8080`. MySQL: `localhost:3306`. Redis: `localhost:6379`.

11. Known Limitations & Next Steps

Payment is mocked. No Stripe keys have been supplied, so `POST /api/bookings/confirm` finalizes the booking immediately with no real charge. Wiring real Stripe would mean: creating a `PaymentIntent` during hold, confirming it client-side, and only finalizing the booking from a webhook (not the client) so a browser closing mid-payment can't create a booking without a successful charge.

Plays, Sports, and Activities are placeholder pages. Building these out fully would mean replicating the entire catalog + booking domain (venue/screen-equivalent, seat-or-ticket inventory, its own admin CRUD) for each category — a significant scope expansion intentionally deferred.

Events are browsable only. No seat/ticket booking model exists for events yet — the `event` table has no relationship to `booking`.

Public movie listing isn't paginated. Only admin list endpoints (movies/venues/shows) are paginated. The public "now showing" list is naturally bounded per city today, but would need pagination if the catalog grows much larger.

No automated tests. All verification in this build was done via manual `curl` checks, compile/boot checks, and browser testing by the user. There is no test suite (unit, integration, or e2e) yet.

Suggested next steps, roughly in priority order

1. Wire real Stripe payments (webhook-driven confirmation, not client-driven)
2. Add a "My Bookings" page on the frontend (the backend endpoint `GET /api/bookings` already exists and is unused by the UI)
3. Add automated tests, starting with the seat-lock concurrency logic (highest-risk correctness surface)
4. Add booking cancellation/refund flow
5. Consider a background job to expire stale `PENDING` bookings if the payment step becomes asynchronous